

TEXT를 사용한 감정 분류

인공지능데이터사이언스학과 24011834 최현서

1. INTRODUCTION

<프로젝트 목표>

- 감정 분석 모델이 직접적인 감정 단어에 얼마나 의존하는지 확인하고자 함
- **text_no_emotion**과 **text_emotion** 비교
- 감정 단어 제거가 성능 및 문맥 기반 분류에 미치는 영향 분석

<데이터셋>

- **CMU - MOSI** :감정 분석을 위한 대표적인 멀티모달 데이터셋
- **MOSI Text Metadata**: 영상 데이터 중 텍스트만 추출한 버전
- 감정 단어 제거 후 성능 비교를 통해 모델의 문맥 이해 가능성 확인

<기대효과>

- 텍스트 감정 분석에서 명시적 단서와 암묵적 단서의 역할을 비교

2. METHODOLOGY

<파이브라인>



< 비교 데이터셋 >

- **text_emotion** : 명시적 감정 단어O => 모델이 더 직접적인 분류 단서 확보
- **text_no_emotion** : 감정 단어 제거O => 문맥 기반 추론 의존도高

< 전처리 및 설정 변경 >

- **max_length**: 50 -> 70 => 문맥 보존 강화
- **min_freq**: 2 -> 1 => 정보 손실 최소화

< 평가 지표 >

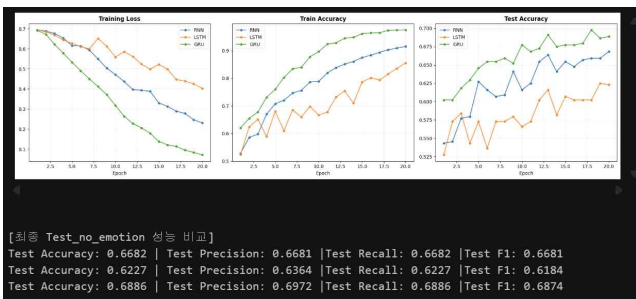
Accuracy, Precision
Recall, F1-score

3. RESULTS

< text_emotion 성능 비교 분석 >



< text_no_emotion 성능 비교 분석 >



< 성능 변화 >

감정 단어 제거 후 모든 모델 **Accuracy, f1-score** 감소
RNN: 중간 수준 감소 / **LSTM**: 가장 큰 감소 / **GRU**: 가장 작은 감소

< 분석 >

- 감정 단어 제거는 전체적인 성능 저하 유발
- 전체적인 모델 성능은 크게 붕괴되지 않음
- **GRU**에서는 안정적인 성능이 파악
- 감정 단어 제거 이후에도 문맥 정보가 일정 수준 보존

4. CODE

< 감정 제거 단어 사전 정의 >

```
# 감정 단어 정의
emotion_words = {
    # positive
    "good", "great", "awesome", "amazing", "excellent", "fantastic",
    "wonderful", "nice", "love", "loved", "like", "liked",
    "fun", "funny", "cool", "best", "favorite", "enjoy", "enjoyed",
    "beautiful", "perfect", "strong", "interesting",

    # negative
    "bad", "terrible", "awful", "horrible", "worst", "boring",
    "hate", "hated", "dislike", "ugly", "annoying", "disappointing",
    "disappointed", "poor", "weak", "stupid", "dumb", "ridiculous",
    "unfunny", "mess", "waste",

    # direct emotion
    "happy", "sad", "angry", "upset", "excited", "depressed",
    "frustrated", "surprised", "shocked", "scared", "afraid",
    "emotional"
}
```

< 텍스트 전처리 및 감정 단어 제거 >

```
# 텍스트에 데이터 전처리 적용 => 1차시 40p
def preprocess_text(text, remove_emotion_words=False):
    text = text.lower() # 소문자 변환
    text = re.sub(r'[\w\s]', '', text) # 특수문자 제거
    tokens = text.split() # 토큰화
    tokens = [token for token in tokens if token not in stop_words] # 불용어 제거

    # 감정 단어 제거
    if remove_emotion_words:
        tokens = [tok for tok in tokens if tok not in emotion_words]
    return tokens

# 감정 단어 제거 전처리
df["tokens_no_emotion"] = df["text"].apply(lambda x: preprocess_text(x, True))
```

< 최소 빈도 기반 vocabulary 생성 >

```
MIN_FREQ = 1 # 본래 값: 2 -> 1 로 수정
# 거의 모든 단어를 살린다 : 잡음 多, 정보를 가장 많이 살리는 방향
vocab_no_emotion = {"<PAD>": 0, "<UNK>": 1}
idx = 2
for word, count in word_counts_no_emotion.items():
    if count >= MIN_FREQ:
        vocab_no_emotion[word] = idx
        idx += 1
```

< 최대 길이 기준 패딩 적용 >

```
MAX_LEN = 70 # 본래 값: 50 -> 70 : 문맥 보존을 더 살리기 위해 파라미터값 변경
PAD_IDX = vocab["<PAD>"]

def pad_sequence(seq, max_len, pad_idx):
    if len(seq) >= max_len:
        return seq[:max_len]
    return seq + [pad_idx] * (max_len - len(seq))
```

< 성능 평가 지표 계산 >

```
# 성능 지표 계산 함수
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
def eval_metrics(model, loader):
    model.eval()
    y_true = []
    y_pred = []
    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            pred = model(xb).argmax(dim=1)
            y_true.extend(yb.cpu().numpy())
            y_pred.extend(pred.cpu().numpy())

    accuracy = accuracy_score(y_true, y_pred) # 정확도 계산
    precision = precision_score(y_true, y_pred, average='weighted') # 정밀도 계산
    recall = recall_score(y_true, y_pred, average='weighted') # 재현율 계산
    f1 = f1_score(y_true, y_pred, average='weighted') # F1 점수 계산
    return accuracy, precision, recall, f1
```

5. CONCLUSION

<결과 해석>

- 감정 단어 제거 후 모델별 성능 하락폭 차이 확인
- 동일한 텍스트라도 모델마다 의존 단서가 다름
- 단순 성능 비교를 넘어 모델별 감정 해석 방식 차이 확인

< 보완점 및 향후 프로젝트 방향성 >

- 감정 단어 제거 강도별 실험 확장
- 오분류 사례 및 클래스별 분석
- 문맥 중심 모델과 비교